

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import ast
import warnings
```

```
In [2]: warnings.filterwarnings('ignore')
```

```
In [3]: Netflix = pd.read_csv('C:\\Users\\visha\\Downloads\\Datasets\\Netflix\\netflix.csv')
Netflix.head()
```

```
Out[3]:
```

	title	type	release_year	age_certification	runtime	genres	production_countries
--	-------	------	--------------	-------------------	---------	--------	----------------------

0	Five Came Back: The Reference Films	SHOW	1945	TV-MA	48	['documentation']	
---	-------------------------------------	------	------	-------	----	-------------------	--

1	Taxi Driver	MOVIE	1976	R	113	['crime', 'drama']	
---	-------------	-------	------	---	-----	--------------------	--

2	Monty Python and the Holy Grail	MOVIE	1975	PG	91	['comedy', 'fantasy']	
---	---------------------------------	-------	------	----	----	-----------------------	--

3	Life of Brian	MOVIE	1979	R	94	['comedy']	
---	---------------	-------	------	---	----	------------	--

4	The Exorcist	MOVIE	1973	R	133	['horror']	
---	--------------	-------	------	---	-----	------------	--



```
In [4]: Netflix.isnull().sum()
```

```
Out[4]: title          1
type          0
release_year    0
age_certification 2610
runtime         0
genres          0
production_countries 0
seasons        3759
imdb_id         444
imdb_score      523
imdb_votes      539
dtype: int64
```

Handling Missing Values

```
In [5]: Netflix.dropna(subset = ['title'], inplace = True)
Netflix['age_certification'] = Netflix['age_certification'].fillna('Not Available')
Netflix['seasons'] = Netflix['seasons'].fillna(0)
Netflix['imdb_id'] = Netflix['imdb_id'].fillna('Not Available')
```

```
Netflix['imdb_score'] = Netflix['imdb_score'].fillna(Netflix['imdb_score'].media
Netflix['imdb_votes'] = Netflix['imdb_votes'].fillna(Netflix['imdb_votes'].media
```

```
In [6]: Netflix.isnull().sum()
```

```
Out[6]: title           0
type           0
release_year    0
age_certification 0
runtime         0
genres          0
production_countries 0
seasons         0
imdb_id         0
imdb_score      0
imdb_votes      0
dtype: int64
```

Genres and Production Country in String like "['crime', 'drama']" so we first convert string to list and After Genres and Production Countries become list. We Explode(One row per genre and Country) because it is easy to count genres and countries

```
In [7]: Netflix['genres'] = Netflix['genres'].apply(ast.literal_eval)
Genre_explode = Netflix.explode('genres')
```

```
In [8]: Netflix['production_countries'] = Netflix['production_countries'].apply(ast.lite
Country_explode = Netflix.explode('production_countries')
```

Export to csv for Genre and Production Country Analysis only for genre and country

```
In [9]: Genre_Explode = Genre_explode.to_csv('C:\\Users\\visha\\Downloads\\Datasets\\Net
Country_Explode = Country_explode.to_csv('C:\\Users\\visha\\Downloads\\Datasets\\
```

```
In [54]: Clean_Netflix = pd.read_csv('C:\\Users\\visha\\Downloads\\Datasets\\Netflix\\Cle
```

```
In [11]: Genre_Explode = pd.read_csv('C:\\Users\\visha\\Downloads\\Datasets\\Netflix\\Gen
Country_Explode = pd.read_csv('C:\\Users\\visha\\Downloads\\Datasets\\Netflix\\C
```

Questions

Total Titles

```
In [12]: Total_Titles = Clean_Netflix['title'].count()
print(f'Total Titles: {Total_Titles}')
```

Total Titles: 5805

Total Shows

```
In [13]: Total_Shows = (Clean_Netflix['type'] == 'SHOW').sum()
print(f'Total Shows: {Total_Shows}')
```

Total Shows: 2047

Total Movies

```
In [14]: Total_Movies = (Clean_Netflix['type'] == 'MOVIE').sum()  
print(f"Total Movies: {Total_Movies}")
```

Total Movies: 3758

Average IMDb Score

```
In [15]: Avg_IMDb_Score = Clean_Netflix['imdb_score'].mean()  
print(f"Average IMDb Score: {Avg_IMDb_Score:.2f}")
```

Average IMDb Score: 6.54

Average Runtime

```
In [16]: Avg_Runtime = Clean_Netflix['runtime'].mean()  
print(f"Average Runtime: {Avg_Runtime:.2f} minutes")
```

Average Runtime: 77.66 minutes

Highest Rated Title

```
In [17]: Highest_Rated_Title = Clean_Netflix.loc[Clean_Netflix['imdb_score'].idxmax(), 't  
print(f" The Highest Rated Title: {Highest_Rated_Title} with an IMDb Score of {C
```

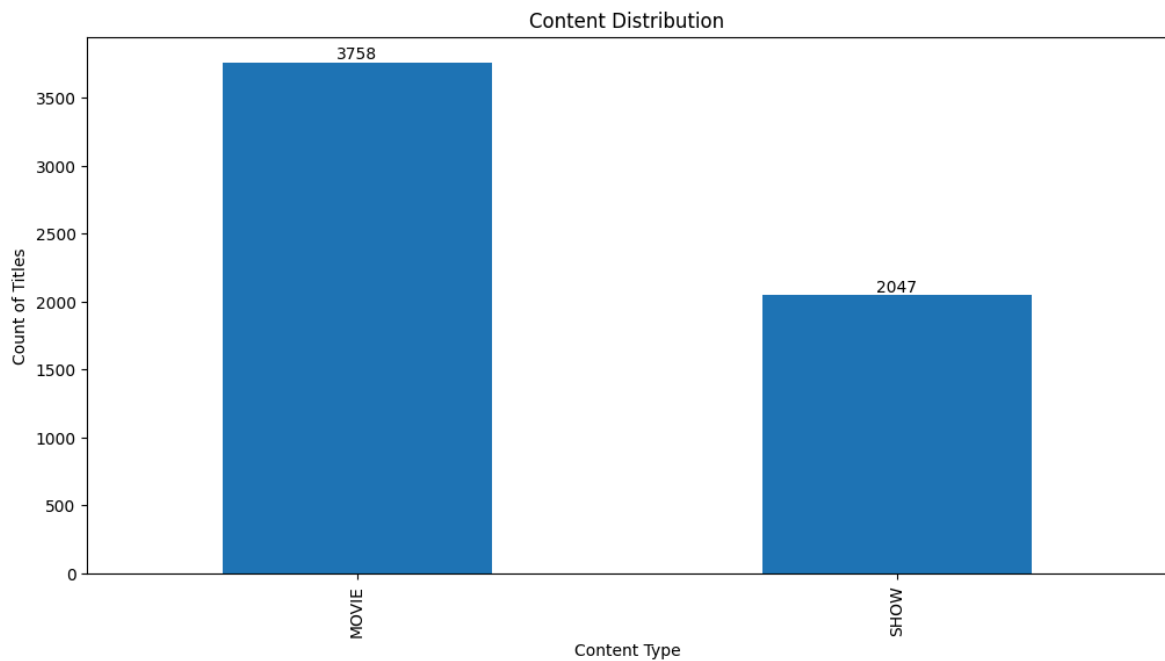
The Highest Rated Title: Khawatir with an IMDb Score of 9.60

Movies vs Shows

```
In [18]: Clean_Netflix['type'].value_counts()
```

```
Out[18]: type  
MOVIE    3758  
SHOW     2047  
Name: count, dtype: int64
```

```
In [19]: Type = Clean_Netflix['type'].value_counts()  
plt.figure(figsize=(12,6))  
Type.plot(kind = 'bar')  
plt.title('Content Distribution')  
plt.xlabel('Content Type')  
plt.ylabel('Count of Titles')  
plt.bar_label(plt.gca().containers[0])  
plt.show()
```



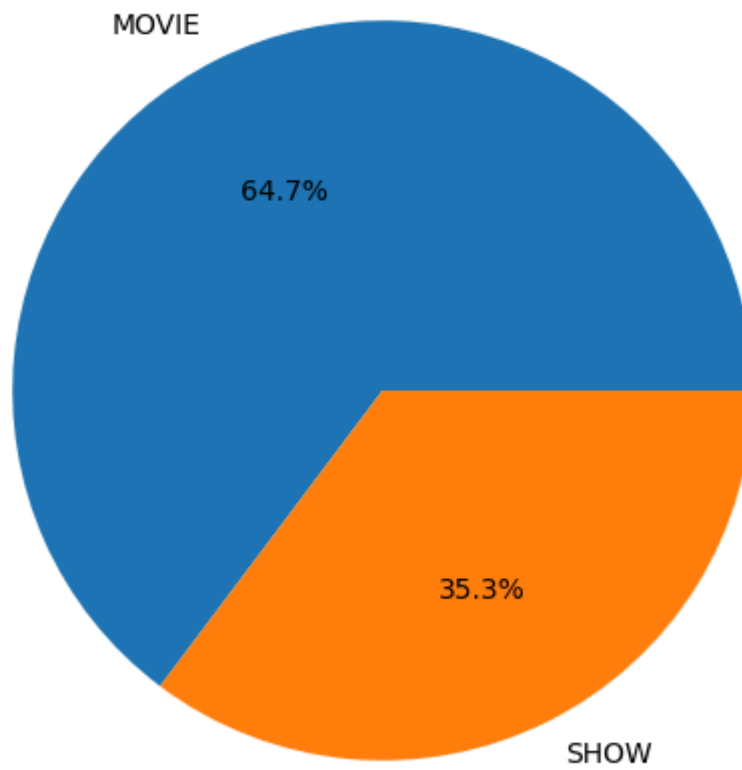
Percentage Split

```
In [20]: Content_Percentage = Clean_Netflix['type'].value_counts(normalize = True)*100  
print(f"Content Percentage: {Content_Percentage.round(2)}")
```

```
Content Percentage: type  
MOVIE      64.74  
SHOW       35.26  
Name: proportion, dtype: float64
```

```
In [21]: Content_Percentage = Clean_Netflix['type'].value_counts()  
  
plt.figure(figsize=(12,6))  
plt.pie(Content_Percentage, labels=Content_Percentage.index, autopct = '%1.1f%%')  
plt.title('Movies vs Shows')  
plt.show()
```

Movies vs Shows



```
In [22]: Country_Explode['production_countries'].isnull().sum()
```

```
Out[22]: np.int64(232)
```

```
In [23]: Country_Explode[Country_Explode['production_countries'].isnull()]
```

Out[23]:

	title	type	release_year	age_certification	runtime	genres	pro
30	Monty Python's Flying Circus	SHOW	1972	TV-MA	43	['comedy']	
295	One Last Shot	MOVIE	1998	Not Available	30	[]	
657	Monty Python Conquers America	MOVIE	2008	Not Available	55	['documentation']	
770	Thomas & Friends: Thomas and the Jet Engine	MOVIE	2004	G	67	['animation']	
776	Wedding Planner	SHOW	2002	TV-14	39	['drama']	
...	...	...	...	...	...	...	...
6705	Loyiso Gola: Unlearning	MOVIE	2021	Not Available	60	['comedy']	
6711	Glimpses of a Future	SHOW	2021	TV-PG	4	[]	
6720	Edis Starlight	MOVIE	2021	Not Available	74	['music', 'documentation']	
6723	Shadow Parties	MOVIE	2021	Not Available	116	['action', 'thriller']	
6724	Mighty Little Bheem: Kite Festival	SHOW	2021	Not Available	0	['family', 'comedy', 'animation']	

232 rows × 11 columns



In [24]: Country_Explode['production_countries'] = Country_Explode['production_countries']

Top 10 Countries

In [25]: Countries = Country_Explode['production_countries'].value_counts().head(10)
print('Top 10 Production Countries:\n', Countries)

Top 10 Production Countries:

production_countries

US 2327

IN 629

GB 406

JP 291

FR 248

[] 232

CA 216

KR 216

ES 212

DE 139

Name: count, dtype: int64

Year Trend

```
In [26]: Clean_Netflix['release_year'].value_counts().sort_index()
```

```
Out[26]: release_year
1945      1
1953      1
1954      2
1956      1
1958      1
...
2018     774
2019     848
2020     805
2021     758
2022     217
Name: count, Length: 67, dtype: int64
```

Most active Year(max Content Release)

```
In [27]: Genre_Explode['genres'].value_counts().sort_values(ascending = False).head(10)
```

```
Out[27]: genres
drama      2901
comedy     2269
thriller   1178
action     1053
romance     958
documentation  910
crime       891
animation   665
fantasy     631
family      622
Name: count, dtype: int64
```

Top Genre

```
In [28]: Genre_Explode['genres'].value_counts().head(10)
```

```
Out[28]: genres
drama      2901
comedy     2269
thriller   1178
action     1053
romance     958
documentation 910
crime       891
animation   665
fantasy     631
family      622
Name: count, dtype: int64
```

Movies vs Shows by Country

```
In [29]: Country_Explode.groupby(['production_countries', 'type']).count().head(10)
```

```
Out[29]:
```

production_countries		type	title	release_year	age_certification	runtime	genres	sea
AE	MOVIE	19	19	19	19	19	19	19
	SHOW	2	2	2	2	2	2	2
AF	MOVIE	2	2	2	2	2	2	2
AL	MOVIE	2	2	2	2	2	2	2
AO	MOVIE	1	1	1	1	1	1	1
AR	MOVIE	53	53	53	53	53	53	53
	SHOW	18	18	18	18	18	18	18
AT	MOVIE	10	10	10	10	10	10	10
	SHOW	3	3	3	3	3	3	3
AU	MOVIE	41	41	41	41	41	41	41

Average Movie Duration

```
In [30]: Clean_Netflix.loc[Clean_Netflix['type'] == 'MOVIE', 'runtime'].mean().round(2)
```

```
Out[30]: np.float64(98.81)
```

Average Show Duration

```
In [31]: Clean_Netflix.loc[Clean_Netflix['type'] == 'SHOW', 'runtime'].mean().round(2)
```

```
Out[31]: np.float64(38.82)
```

Longest Movie Runtime

```
In [32]: Clean_Netflix.loc[Clean_Netflix['runtime'].idxmax()]
```



```
Out[32]: title           The School of Mischief
type                MOVIE
release_year        1973
age_certification    Not Available
runtime             251
genres              ['comedy']
production_countries ['EG']
seasons             0.0
imdb_id             Not Available
imdb_score           6.6
imdb_votes           2279.0
Name: 34, dtype: object
```

Rating Distribution

```
In [33]: Netflix['imdb_score'].value_counts()
```

```
Out[33]: imdb_score
6.6      723
6.8      199
6.5      193
6.2      192
7.4      190
...
1.9        1
1.5        1
2.4        1
1.6        1
1.8        1
Name: count, Length: 81, dtype: int64
```

Top Rating by content type

```
In [34]: Netflix.groupby(['type', 'imdb_score'])['title'].count().sort_values(ascending=F
```

```
Out[34]: type  imdb_score
MOVIE  6.6         485
SHOW   6.6         238
MOVIE  6.2         139
        6.3         137
        6.5         136
...
SHOW   1.7          1
        2.6          1
        3.6          1
        3.1          1
        9.5          1
Name: title, Length: 141, dtype: int64
```

Content Growth Per Year

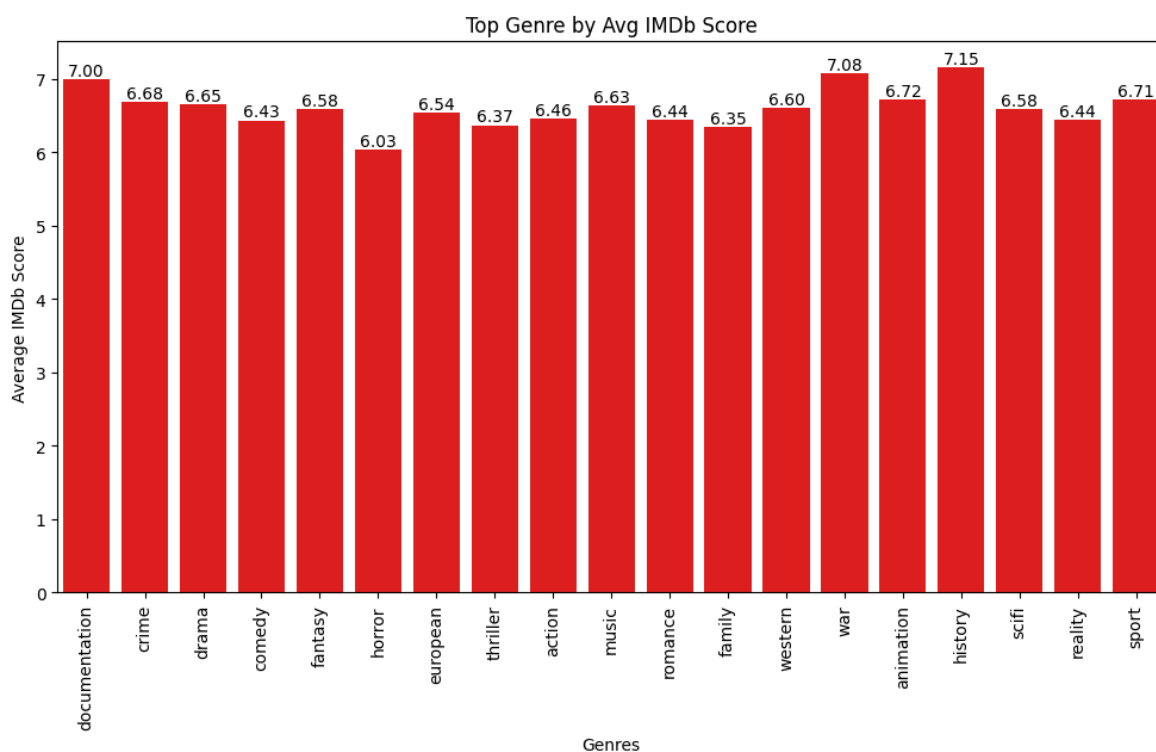
```
In [35]: Clean_Netflix.groupby('release_year')['title'].count().pct_change()*100
```

```
Out[35]: release_year
1945      NaN
1953      0.000000
1954     100.000000
1956     -50.000000
1958      0.000000
...
2018     33.448276
2019      9.560724
2020     -5.070755
2021     -5.838509
2022    -71.372032
Name: title, Length: 67, dtype: float64
```

Top Genre by Average IMDb Score

```
In [36]: Genre_Average = Genre_Explode.groupby('genres')['imdb_score'].mean().round(2).sort_values(ascending=False)

plt.figure(figsize=(12,6))
sns.barplot(data=Genre_Explode, x = 'genres', y = 'imdb_score', color = 'red', edgecolor='black')
plt.title('Top Genre by Avg IMDb Score')
plt.xlabel('Genres')
plt.ylabel('Average IMDb Score')
plt.bar_label(plt.gca().containers[0], fmt = '%.2f')
plt.xticks(rotation = 90)
plt.show()
```



Top Country by Average IMDb Score (minimum 20 titles)

```
In [37]: Country_Stats = (
    Country_Explode.groupby('production_countries')['imdb_score']
    .agg(mean_score='mean', title_count='count')
    .reset_index()
)
Country_Average = (
    Country_Stats[Country_Stats['title_count'] >= 20]
```

```

        .sort_values(by='mean_score', ascending=False)
        .head(10)
    )
Country_Average

```

Out[37]:

	production_countries	mean_score	title_count
55	KR	7.135648	216
51	JP	6.963574	291
78	PS	6.914286	21
25	DK	6.910345	29
19	CN	6.805769	104
70	NO	6.790909	22
32	GB	6.778571	406
95	TW	6.750820	61
57	LB	6.744444	36
6	AU	6.718072	83

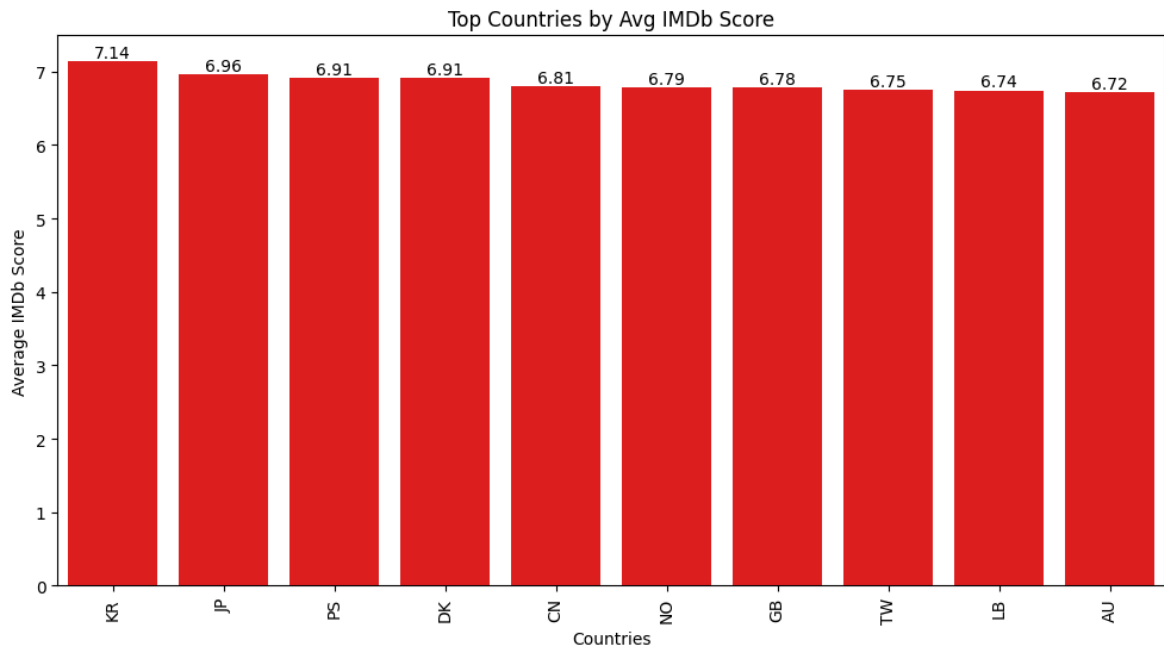
In [38]:

```

Country_Stats = (
    Country_Explode.groupby('production_countries')['imdb_score']
    .agg(mean_score='mean', title_count='count')
    .reset_index()
)
Country_Average = (
    Country_Stats[Country_Stats['title_count'] >= 20]
    .sort_values(by='mean_score', ascending=False)
    .head(10)
)

plt.figure(figsize=(12,6))
sns.barplot(data= Country_Average , x = 'production_countries', y = 'mean_score')
plt.title('Top Countries by Avg IMDb Score')
plt.xlabel('Countries')
plt.ylabel('Average IMDb Score')
plt.bar_label(plt.gca().containers[0], fmt = '%.2f')
plt.xticks(rotation = 90)
plt.show()

```



Genre Distribution for Movies Only

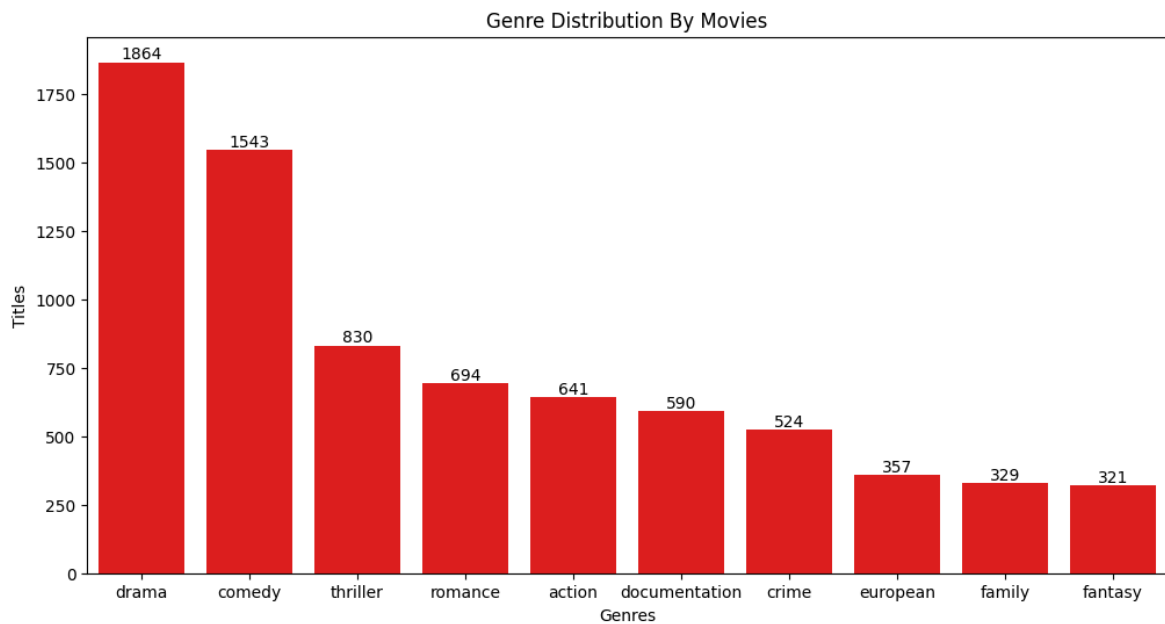
```
In [39]: Genre_Stats = (
    Genre_Explode.groupby(['genres', 'type'])['title']
    .agg(title_count='count')
    .reset_index()
)
Movie_genre = (Genre_Stats[Genre_Stats['type'] == 'MOVIE']).sort_values(by = 'title_count', ascending=False)
Movie_genre
```

Out[39]:

	genres	type	title_count
10	drama	MOVIE	1864
4	comedy	MOVIE	1543
32	thriller	MOVIE	830
26	romance	MOVIE	694
0	action	MOVIE	641
8	documentation	MOVIE	590
6	crime	MOVIE	524
12	european	MOVIE	357
14	family	MOVIE	329
16	fantasy	MOVIE	321

```
In [40]: Genre_Stats = (
    Genre_Explode.groupby(['genres', 'type'])['title']
    .agg(title_count='count')
    .reset_index()
)
Movie_genre = (Genre_Stats[Genre_Stats['type'] == 'MOVIE']).sort_values(by = 'title_count', ascending=False)
plt.figure(figsize=(12,6))
```

```
sns.barplot(data = Movie_genre, x = 'genres', y = 'title_count', color = 'red',
plt.title('Genre Distribution By Movies')
plt.xlabel('Genres')
plt.ylabel('Titles')
plt.bar_label(plt.gca().containers[0])
plt.show()
```



Genre Distribution for Shows Only

```
In [41]: Genre_Stats = (
    Genre_Explode.groupby(['genres', 'type'])['title']
    .agg(title_count='count')
    .reset_index()
)
Shows_genre = (Genre_Stats[Genre_Stats['type'] == 'SHOW'].sort_values(by = 'title_count', ascending=False))
Shows_genre
```

```
Out[41]:
```

	genres	type	title_count
11	drama	SHOW	1037
5	comedy	SHOW	726
1	action	SHOW	412
3	animation	SHOW	394
29	scifi	SHOW	377
7	crime	SHOW	367
33	thriller	SHOW	348
9	documentation	SHOW	320
17	fantasy	SHOW	310
15	family	SHOW	293

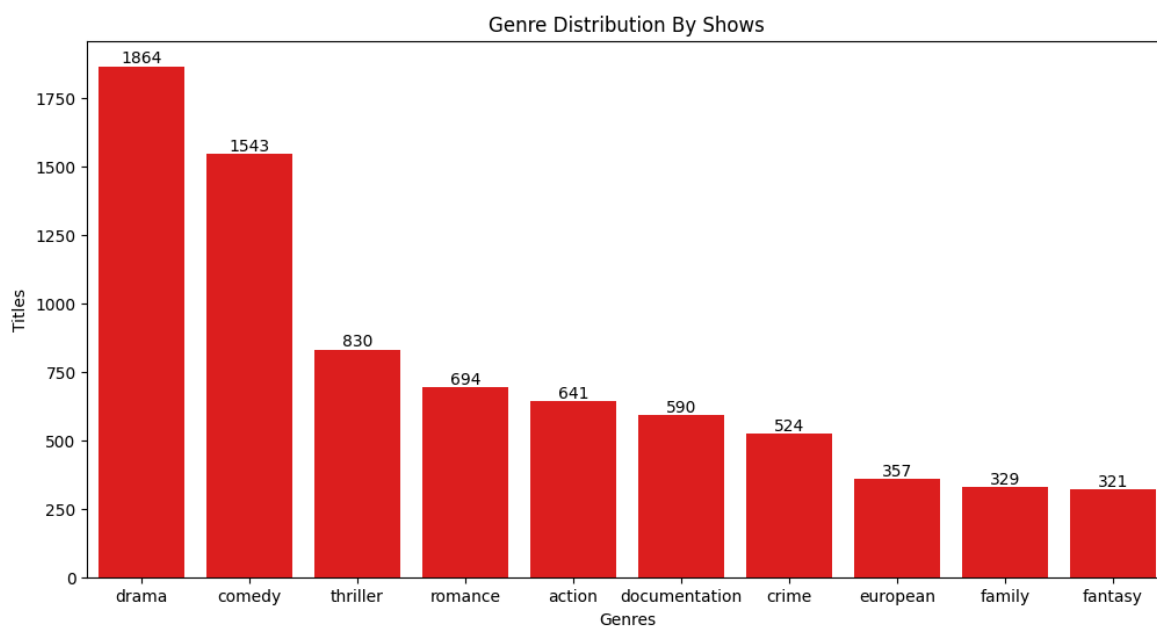
```
In [42]: Genre_Stats = (
    Genre_Explode.groupby(['genres', 'type'])['title']
```

```

        .agg(title_count='count')
        .reset_index()
    )
    Show_genre = (Genre_Stats[Genre_Stats['type'] == 'SHOW'].sort_values(by = 'title_count', ascending=False))

    plt.figure(figsize=(12,6))
    sns.barplot(data = Movie_genre, x = 'genres', y = 'title_count', color = 'red',
    plt.title('Genre Distribution By Shows')
    plt.xlabel('Genres')
    plt.ylabel('Titles')
    plt.bar_label(plt.gca().containers[0])
    plt.show()

```



Find which country has the most titles with IMDb score greater than 8

```

In [45]: Country_title = (
    Country_Explode[Country_Explode['imdb_score'] > 8 ]
    .groupby('production_countries')['title']
    .nunique()
    .sort_values(ascending=False)
).head(10)
Country_title

```

```

Out[45]: production_countries
US      145
IN       54
GB       47
KR       38
JP       34
[]       14
CN       11
AU        9
CA        9
CO         6
Name: title, dtype: int64

```

```

In [51]: Country_title = (
    Country_Explode[Country_Explode['imdb_score'] > 8 ]
    .groupby('production_countries')['title']
    .nunique()

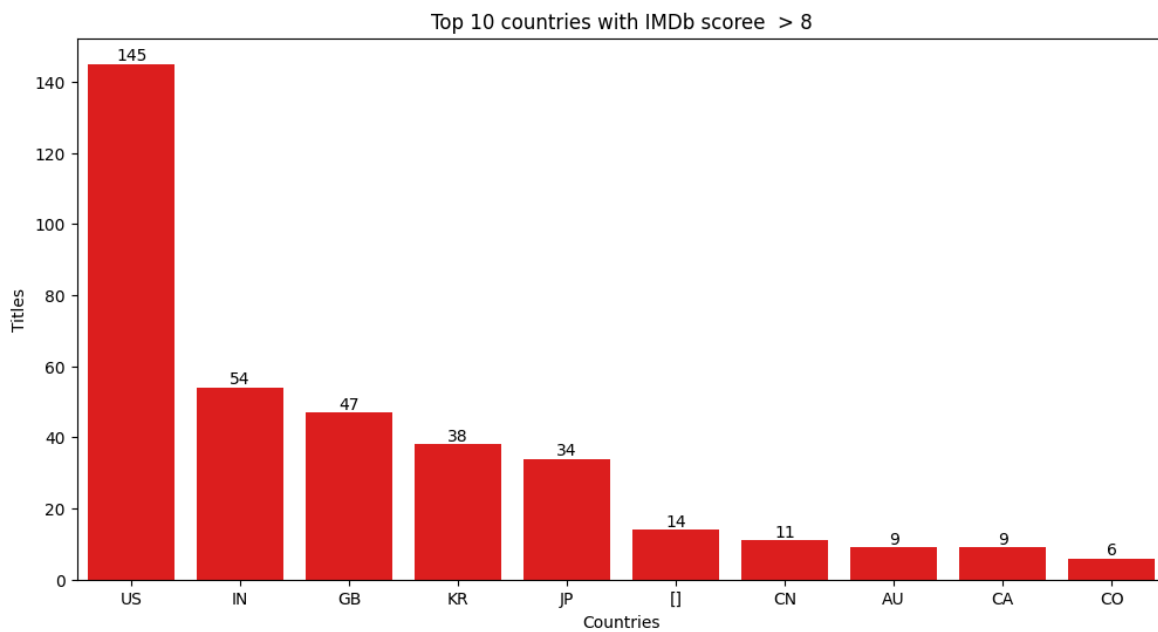
```

```

        .sort_values(ascending=False)
        .reset_index()
    ).head(10)

plt.figure(figsize=(12,6))
sns.barplot(data = Country_title, x = 'production_countries', y = 'title', color = 'red')
plt.title('Top 10 countries with IMDb score > 8')
plt.xlabel('Countries')
plt.ylabel('Titles')
plt.bar_label(plt.gca().containers[0])
plt.show()

```



Check whether runtime and IMDb score are correlated

```
In [55]: Clean_Netflix[['runtime', 'imdb_score']].corr()
```

```
Out[55]:
```

	runtime	imdb_score
runtime	1.000000	-0.151957
imdb_score	-0.151957	1.000000

```

In [59]: plt.figure(figsize=(8,6))

plt.scatter(Clean_Netflix['runtime'], Clean_Netflix['imdb_score'], color = 'Red')

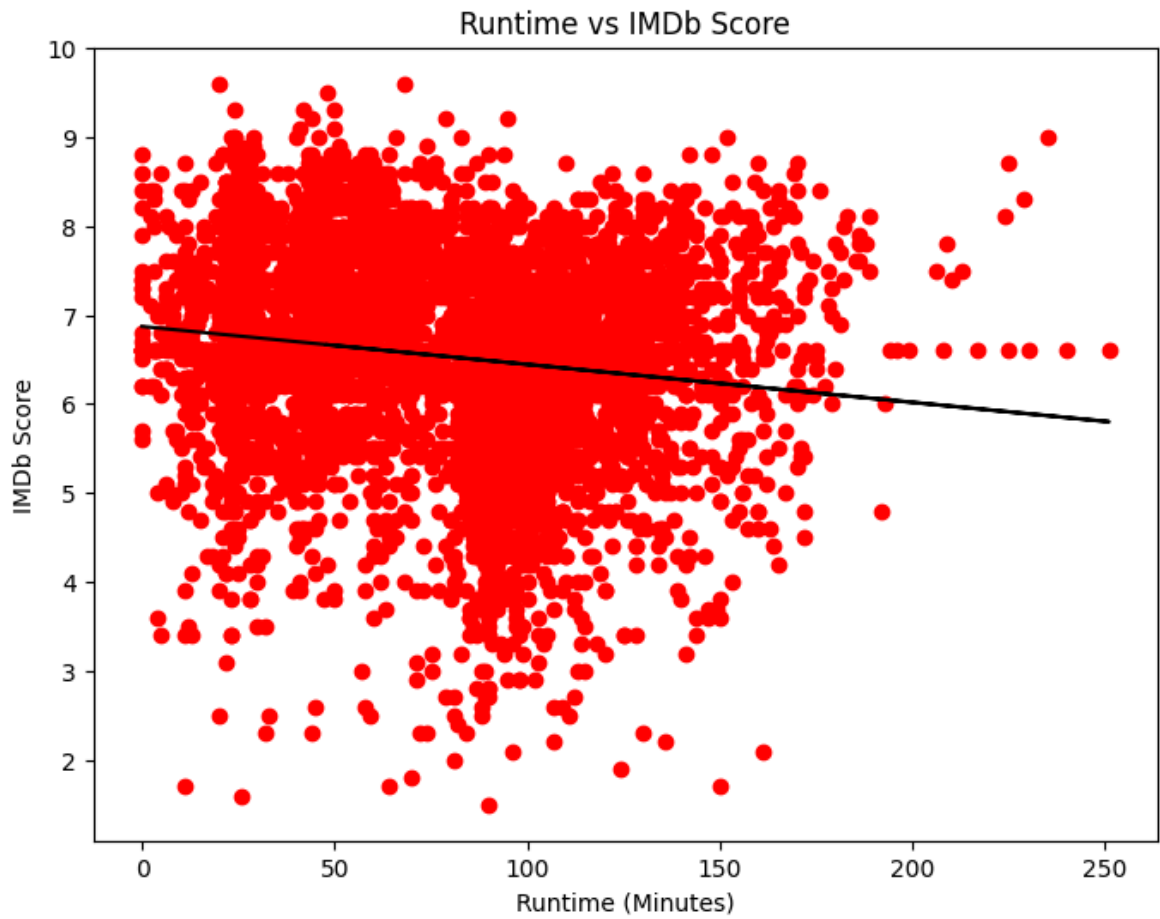
z = np.polyfit(Clean_Netflix['runtime'], Clean_Netflix['imdb_score'], 1)
p = np.poly1d(z)

plt.plot(Clean_Netflix['runtime'], p(Clean_Netflix['runtime']), color = 'Black')

plt.title('Runtime vs IMDb Score')
plt.xlabel('Runtime (Minutes)')
plt.ylabel('IMDb Score')

plt.show()

```



Runtime has little impact on IMDb ratings. Longer content is not necessarily rated higher than shorter content